Lab#3



– Digital Electronics –

On Chip Memory and IPs

This purpose of this lab is practicing with on-chip Memory and other IPs.

Contents:
1. Design of a VGA controller

Abbreviations and acronyms:
IC – Integrated Circuit
LE – Logic Element
LED – Light Emitting Diode
LUT – Look Up Table
MUX – Multiplexer
RTL – Register Transfer Level

1 – Design of a VGA controller

Design and implement a circuit that acts as a VGA controller with the following specifications:

1. Resolution 640x480 @ 60 Hz.
2. The inputs of the circuit are: the 50 MHz clock (`CLOCK_50`) and an asynchronous reset (KEY(0)).
3. The outputs of the circuit are the inputs to the ADV7123 DAC, available on the DE1-SoC board. Please refer to paragraph 3.6.6 (VGA) of the DE1-SoC user manual and to the ADV7123 datasheet for details.

Suggestion: a simple controller can be obtained using two counters (horizontal/vertical), some comparators (back/front porch, active) and some glue logic.

The circuit must draw on the screen four rectangles as shown in Fig. 1

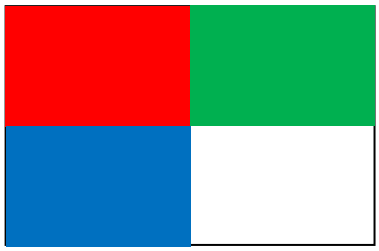


Fig. 1 four-rectangles image.

Implement the circuit according with the following steps.

1. **Design and simulation**
   a. **Understand** the function of each output and how each output should be driven:
      i. Find in the DE1-SoC user manual the pixel clock frequency, which will be both the controller clock frequency and the VGA clock frequency.
      ii. Understand the shape of the Horizontal and Vertical synchronization signals. Even if values reported in the DE1-SoC user manual (Table 3-14) are approximated, the system will work correctly. Pay attention to the fact that VGA_HS and VGA_VS signal are directly connected to the VGA connector, whereas the other signals are connected to the ADV7123 (one clock cycle of pipeline).
      iii. `BLANK` and `SYNC` signals are not used in this project, but they have to be driven correctly (check the ADV7123 datasheet).
      iv. Understand how to drive the `R`, `G` and `B` signals.
   b. **Draw** a block scheme of the circuit.
   c. **Draw** the timing diagram of the circuit and (if needed) refine the circuit.
   d. **When you are confident of the correctness of your circuit write** the HDL, simulate it and verify it behaves correctly. During this step and for simulation purpose only you can generate the pixel clock from the `CLOCK_50` with some logic.
   e. Ask an AI to generate the same functionality you implemented (with the same HDL you employed) and compare the result.

   Suggestion: design your circuit to be parametric with respect to the number of bits and the thresholds you have to set. Thus, you can verify your circuit with a short simulation by changing these parameters to small values.

2. **Implementation on the DE1-SoC board**
   Generating a clock with logic is not a good design practice. So instead of deriving the pixel clock from the `CLOCK_50` with some logic, it is better to obtain it through a Phase-Locked-Loop (PLL) component. PLLs are available inside the FPGA so we can use them. They are provided inside Quartus-Prime as Intellectual Properties (IPs). The PLL allows to generate a clock signal at a certain frequency starting from a reference clock. The

generated clock can be either higher or lower than the reference clock, depending on the design requirements. In particular, the PLL allows not only to achieve a certain frequency but also to obtain a generated clock with a specific phase with respect to the reference clock. Moreover, one PLL can generate more clocks starting from the same reference clock.

Start creating a new Quartus-Prime project, which includes your VGA controller. Remember to select the correct device for the DE1-SoC board and to import the .qsf file with the right pin mapping.

On the right side of your project (inside Quartus-Prime) there is window named "IP Catalog". In the IP Catalog you can find **Library->Basic Functions->Clocks**; **PLLs and Resets->PLL.** Double click the Altera PLL and a window will open. This is the first step to customize the PLL IP for your design. Choose HDL file type you prefer and put in the folder containing your project an "IP variation name file" named (as an example) mypll.v/mypll.vhd.

This step opens the MegaWizard, an environment for IP customization, so you can specify what you need for your PLL. The default configuration needs some changes for our purpose:

a. In the "General" tab the "Reference Clock Frequency" must be 50.0 MHz.
b. The number of "Output Clocks" must be 2 (the explanation will be given later on).
c. The first generated clock "outclk0" must have as "Desired Frequency" the value you need for your circuit (e.g. 25.0 MHz) with a "Phase Shift" equal to 0 (independently of the unit) and a "Duty Cycle" at 50%.
d. The second generated clock "outclk1" must be equal to "outclk0" but with a phase shift of 180°, namely the "Desired Frequency" must be same one as "outclk0", the "Phase Shift units" must be "degrees", the "Phase Shift" must be "180.0 degrees " and the "Duty Cycle" at 50%.
e. All the other parameters can be left to their default values. For further details about Altera PLLs please refer to the "Embedded Peripherals IP User Guide".

As shown in the left part of the MegaWizard window the PLL we are generating has two inputs (`refclk` and `reset`) and three outputs (`outclk_0`, `outclk_1` and `locked`):

- `refclk` is the reference clock, namely the clock given as an input to the PLL, in our case it is the `CLOCK_50`.
- `reset` is the input reset of the PLL it is active high: the reset is asserted with '1'.
- `outclk_0` and `outclk_1` are the generated clocks.
- `locked` is an output produced by the PLL to notify when generated clocks are stable. It is active high and it is better to keep a circuit using a PLL in a safe state (such as in reset) as long as the PLL is not ready (as long as locked is '0').

Now you can click on "Finish", the MegaWizard will generate the PLL. If everything is ok you will see in the Generation window "Generation successful" and you can click on "Exit".

Now Quartus-Prime prompts to add the PLL IP to your project. Answering yes the .qip file (Quartus IP file) will be added to your project, otherwise you have to add it manually. If you browse the directory containing your project you will find the file mypll.v/`mypll.vhd`, which has been automatically generated by the MegaWizard. This file allows you to check that the interface of your PLL is the one you wanted.

```
// Verilog
module mypll (
    input  refclk,    // refclk.clk
    input  rst,       // reset.reset
    output outclk_0,  // outclk0.clk
    output outclk_1,  // outclk1.clk
    output locked     // locked.export
);
endmodule
```

```
-- VHDL
entity mypll is
      port (
            refclk   : in  std_logic := '0'; --  refclk.clk
            rst      : in  std_logic := '0'; --   reset.reset
            outclk_0 : out std_logic;        -- outclk0.clk
            outclk_1 : out std_logic;        -- outclk1.clk
            locked   : out std_logic         --  locked.export
      );
end entity mypll;
```

Why should we need `outclk_1` ? To put the display in the best condition to correctly sample the data (see Fig. 2).

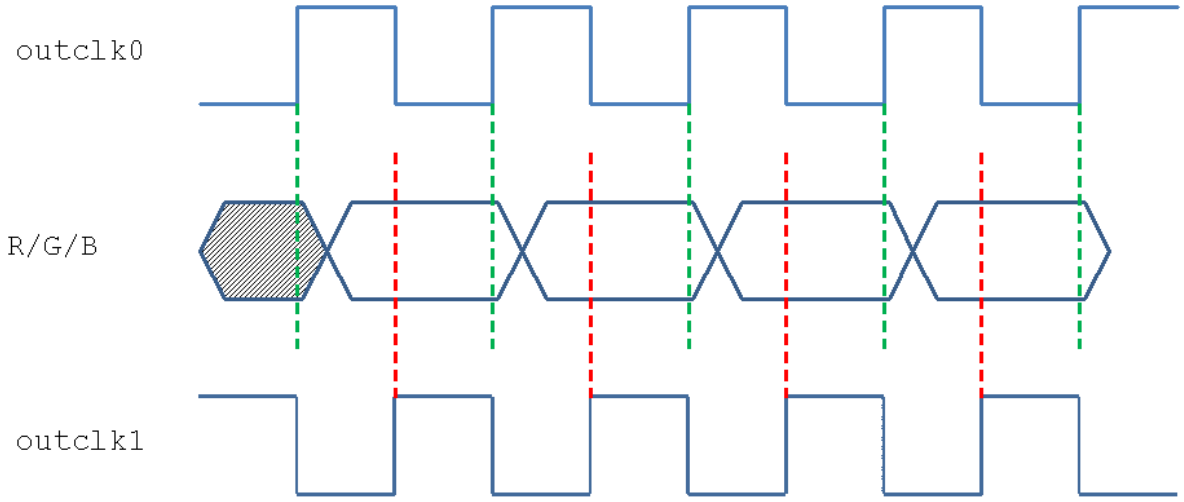


Fig.2 Timing of `outclk0` and `outclk1` with respect to the pixel value.

Indeed, `outclk_1` has a lot of margin to correctly sample the data (it samples the data nearly at half the period). Thus, `outclk_0` will be the internal clock of the controller and `outclk_1` the `VGA_CLK`. Fig. 3 shows how to connect the PLL inside the controller.

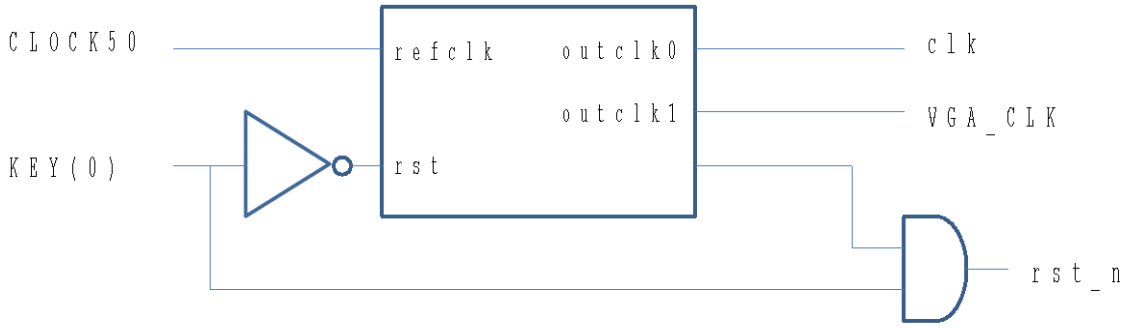


Fig. 3 Connecting the PLL to the circuit.

You can simulate your design with the instantiated PLL with ModelSim/QuestaSim FPGA Altera Starter Edition. In order to have a model for simulating the PLL you have to add to your ModelSim/QuestaSim project the simulation model (automatically generated by the MegaWizard). The simulation model is located in the directory named `mypll_sim`, which is placed inside your project directory. The file containing the simulation model is `mypll.vo/mypll.vho`.

When you are ready to test your controller with the PLL on the board remember to add to the project the .sdc file, which must contain the constraints for all the clocks in your design. Then,

a. **Run** the compilation of your project and check that it finishes correctly.
b. **Check** the RTL view of the design and compare it with your block scheme**.**
c. **Test** your project by connecting the VGA connector from the DE1-SoC board to a VGA monitor.

2 – Display a picture on the monitor

In order to display a picture on the monitor you need a place to store the picture and then some logic to read it and sent it to the monitor. We store the picture into a memory. For the sake of simplicity we consider an RGB picture, so we need three memories, one for each color component. The amount of memory required depends on the image itself. Let us assume that we want to display a frame taken from a QCIF video sequence, the size of each frame is 176x144 pixels and each pixel requires 24 bit (8 bit for R, 8 bit for G and 8 bit for B). You can find on "Portale della didattica" a sample sequence, named tulips_rgb444_prog_planar_qcif.yuv.

As memories are available on the FPGA we can simply use them. A simple way to use a memory is to create a custom IP through the MegaWizard. On the right side of your project (inside Quartus-Prime) in the "IP Catalog" window select **Library->Basic Functions->On Chip Memory->ROM: 1 port.** Double click the ROM and a window will open. Choose VHDL as the file type and put in the folder containing your project an "IP variation name file" named (as an example) "img_rom_r.vhd". The MegaWizard environment will open and you can customize you memory. First choose the width, namely the amount of bits for each value in the memory (which is also the width of the output port of the memory). Since each component of a pixel is represented on 8 bits you have to choose 8. Then, you have to choose the depth of the memory, namely the number of word stored in the memory. Since one frame contains 176x144 pixels the number of words must be at least … (not too difficult). You can use the default settings for the block type (Auto) and for the clocking method (Single clock). Then, click "Next".

As shown in the picture on the left-hand-side of the MegaWizard window the address of the memory is (always) registered, whereas the output port can be either registered or not (check/uncheck the box 'q' output port). Depending on the behavior and the timing of your circuit you can choose if you need registered output or not. If you want you can create a clock enable (not needed for this project). Moreover, you can create an asynchronous reset ('aclr') for all registered ports. This is recommended so you have all your registers with an asynchronous reset for correct initialization. Finally, you can create read enable signal ('rden'). In this application the read enable signal is particularly useful to be sure that data are read from the memory with the correct timing. Then, click "Next".

Since we want to use the memory as a ROM, we need to put into the memory the proper content. This can be done through the MegaWizard by providing a file (with proper format) containing the data. For the sake of simplicity you can find on "Portale della didattica" the file "tulips_rgb444_prog_planar_qcif_frame1_R.mif", which is ready to use. If you use this already available file, you have to specify it through the MegaWizard by clicking on the "Browse" button. Once ready, click "Next".

Finally, you can simply click "Next" and then "Finish". The system suggests you to include the new IP to project, otherwise you have to do it by hand.

Repeat the same steps for the other two memories (for G and B components).

As a result, you can see that in the directory of your project you have an HDL model for each ROM. As an example:

```
// Verilog
module img_rom_r (
    input  aclr,
    input  [7:0]  address,
    input  clock,
    input  rden,
    output [7:0]  q
);
endmodule

-- VHDL
ENTITY img_rom_r IS
      PORT
      (
            aclr        : IN STD_LOGIC  := '0';
            address     : IN STD_LOGIC_VECTOR (7 DOWNTO 0);
            clock       : IN STD_LOGIC  := '1';
            rden        : IN STD_LOGIC  := '1';
            q           : OUT STD_LOGIC_VECTOR (7 DOWNTO 0)
      );
END img_rom_r;
```

Note: the component generated by the MegaWizard, instantiates an altsyncram block, which has a generic (init_file) representing the name of the .mif file. Thus, instead of generating three memories for R, G, and B, you can consider modifying the ROM model interface to have just one component (e.g img_rom_gen), such that you can specify at instantiation-time the name of the .mif file.

The next step is to add the components in your HDL, to instantiate and connect them to your circuit.

Now you can simulate your complete design with with ModelSim/QuestaSim Altera FPGA Starter Edition.

Finally, when you are ready to test your complete controller on the board you can:

a. **Run** the compilation of your project and check that it finishes correctly.
b. **Test** your project by connecting the VGA connector from the DE1-SoC board to a VGA monitor.

3 – Display a sequence on the monitor

Modify the circuit to instantiate the memories required to store four frames of the “tulips_rgb444_prog_planar_qcif” sequence (the .mif files are already available on portale). Then, by properly choosing which frame to display, you can display the different frames on the monitor. Set the frame rate to about 25 frames per second, use the switches to see the effect of increasing and decreasing the frame rate.

a. **Simulate** your design with ModelSim/QuestaSim Altera FPGA Starter Edition.
b. **Run** the compilation of your project and check that it finishes correctly.
c. **Test** your project by connecting the VGA connector from the DE1-SoC board to a VGA monitor.

Appendix

Further details about VGA signals are available here:

- Precise VGA parameters for different formats, http://tinyvga.com/vga-timing

For further details about the .mif format you can refer to the Intel-Altera documentation:

- https://www.intel.com/content/www/us/en/programmable/quartushelp/17.0/reference/glossary/def_mif.htm